

LAPORAN PROJECT
PEMROSESAN PARALLEL
SIMPLE ADDITIVE WEIGHTING (SAW)

Dosen Pengampu: Heri Wijayanto, S.T., M.T., Ph.D.



DISUSUN OLEH:

Alya Rasyida Putri	(F1D02410002)
Sabrina Mawadathun Salsabila	(F1D02410092)
Felyvia Trevina Gunawan	(F1D02410112)
Nadine Desvita Alifia Putri	(F1D02410129)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MATARAM

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan project berjudul “*Simple Additive Weighting* (SAW)”, yang disusun dalam rangka memenuhi tugas besar mata kuliah Pemrosesan Paralel Kelas C Tahun 2026. Penulis menyadari bahwa laporan ini tidak akan dapat tersusun dengan baik tanpa adanya bimbingan, arahan, serta dukungan dari berbagai pihak. Oleh karena itu, penulis menyampaikan rasa terima kasih yang sebesar-besarnya kepada Bapak Heri Wijayanto S.T., M.T., Ph.D. selaku dosen pengampu mata kuliah, atas segala bantuan, bimbingan, dan masukan berharga yang telah diberikan selama proses pengerjaan project ini.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan dan belum mencapai kesempurnaan. Oleh karena itu, penulis sangat mengharapkan adanya kritik dan saran yang bersifat membangun sebagai bahan evaluasi untuk perbaikan di masa mendatang. Penulis berharap, melalui penerapan metode *Simple Additive Weighting* (SAW) dalam sistem yang dikembangkan, dapat membantu proses pengambilan keputusan, sehingga menjadi lebih objektif, efektif, dan efisien. Akhir kata, semoga laporan ini dapat memberikan manfaat bagi para pembaca serta menjadi salah satu kontribusi dalam pengembangan ilmu pengetahuan, terutama di bidang sistem pendukung keputusan dan pemrosesan paralel.

Mataram, April 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat menuntut adanya sistem yang mampu mengolah data secara cepat, akurat, dan efisien. Dalam berbagai bidang, terutama dalam pengambilan keputusan, sering kali dihadapkan pada permasalahan yang melibatkan banyak kriteria dan alternatif. Untuk mengatasi hal tersebut, diperlukan suatu metode yang dapat membantu proses pengambilan keputusan secara sistematis dan terstruktur.

Salah satu metode yang banyak digunakan dalam sistem pendukung keputusan adalah metode *Simple Additive Weighting* (SAW). Metode ini sangat relevan untuk berbagai bidang, terutama dalam sistem rekomendasi, karena mampu membantu pengguna dalam menentukan pilihan terbaik dari banyak alternatif yang tersedia. Permasalahan yang sering muncul adalah banyaknya pilihan yang membuat pengguna kesulitan dalam mengambil keputusan secara tepat. Oleh karena itu, diperlukan suatu sistem yang dapat melakukan seleksi alternatif secara objektif berdasarkan sejumlah kriteria yang relevan.

Metode SAW bekerja dengan menyeleksi alternatif terbaik dari beberapa pilihan berdasarkan kriteria yang telah ditentukan, kemudian melakukan proses perankingan untuk mengetahui alternatif dengan nilai tertinggi hingga terendah. Konsep dasar dari metode SAW adalah menghitung penjumlahan terbobot dari setiap alternatif pada seluruh atribut yang ada. Dengan pendekatan tersebut, metode SAW dapat memberikan hasil perankingan yang sistematis dan mendukung proses pengambilan keputusan yang membantu pengguna dalam menentukan pilihan terbaik secara tepat.

1.2 Tujuan

Penelitian ini dilatarbelakangi oleh kebutuhan pengambilan keputusan multikriteria yang melibatkan banyak alternatif dan kriteria. Untuk mengatasi hal tersebut, digunakan metode *Simple Additive Weighting* (SAW) yang membantu pengambilan keputusan secara sistematis dan efisien. Oleh karena itu, tujuan dari penelitian ini adalah:

1. Menerapkan metode *Simple Additive Weighting* (SAW) dalam proses pengambilan keputusan multikriteria.
2. Menghitung dan menentukan alternatif terbaik berdasarkan kriteria yang telah ditentukan.
3. Menganalisis hasil perankingan yang diperoleh menggunakan metode SAW.

1.3 Manfaat

Penelitian ini diharapkan dapat memberikan kontribusi baik secara teoritis maupun praktis dalam bidang komputasi paralel dan pengambilan keputusan berbasis data. Berdasarkan tujuan yang telah dirumuskan, terdapat sejumlah manfaat yang dapat diperoleh dari penelitian ini, antara lain:

1. Memberikan pemahaman mengenai penerapan metode *Simple Additive Weighting* (SAW) dalam sistem pendukung keputusan, serta integrasinya dengan komputasi paralel untuk menyelesaikan permasalahan pengambilan keputusan multikriteria.
2. Meningkatkan efisiensi waktu pemrosesan data melalui penerapan komputasi paralel dalam perhitungan metode SAW, sehingga proses pengambilan keputusan dapat dilakukan dengan lebih cepat terutama pada data berukuran besar.
3. Membantu pengguna dalam memperoleh rekomendasi alternatif terbaik secara lebih cepat dan akurat berdasarkan kriteria yang telah ditentukan, melalui hasil perankingan yang dihasilkan oleh metode SAW.

BAB II

DASAR TEORI DAN TINJAUAN PUSTAKA

2.1 Pengertian Dan Fungsi *Simple Additive Weighting* (SAW)

Metode *Simple Additive Weighting* (SAW) merupakan salah satu teknik dalam bidang Sistem Pendukung Keputusan (SPK) yang digunakan untuk membantu proses pengambilan keputusan secara sistematis dan objektif. SAW sering disebut sebagai metode penjumlahan terbobot, karena konsep dasarnya adalah menjumlahkan nilai setiap alternatif yang telah dikalikan dengan bobot dari masing-masing kriteria.

Dalam metode SAW, setiap alternatif akan dinilai berdasarkan sejumlah kriteria yang telah ditentukan sebelumnya. Nilai-nilai tersebut kemudian dinormalisasi agar dapat dibandingkan secara adil, terutama jika memiliki skala yang berbeda. Setelah proses normalisasi, setiap nilai dikalikan dengan bobot kepentingan kriteria, lalu dijumlahkan untuk memperoleh nilai akhir. Alternatif dengan nilai tertinggi dianggap sebagai pilihan terbaik.

Secara matematis, konsep SAW berkaitan dengan proses normalisasi dan pembobotan dalam kerangka *Multi-Criteria Decision Making* (MCDM), yaitu pengambilan keputusan yang melibatkan banyak kriteria.

Adapun fungsi utama metode SAW adalah sebagai berikut:

- Membantu pengambilan keputusan yang melibatkan banyak kriteria secara objektif.
- Menyederhanakan proses penilaian berbagai alternatif.
- Memberikan peringkat (*ranking*) terhadap alternatif terbaik.
- Mengurangi subjektivitas dalam proses pemilihan.
- Mempermudah perbandingan antar alternatif berdasarkan nilai numerik.

Metode SAW banyak digunakan dalam berbagai bidang, seperti seleksi penerima beasiswa, penentuan karyawan terbaik, pemilihan lokasi strategis, hingga sistem rekomendasi, karena perhitungannya yang sederhana namun efektif.

2.2 Pengertian MPI (*Message Passing Interface*)

Message Passing Interface (MPI) merupakan standar dalam pemrograman paralel yang menggunakan konsep pertukaran pesan. Melalui MPI, sejumlah proses komputasi yang berjalan secara simultan, baik pada satu maupun beberapa komputer, dapat saling berinteraksi dengan cara mengirim dan menerima pesan. MPI menyediakan berbagai fungsi komunikasi tingkat tinggi, sehingga pengembang tidak perlu mengelola secara langsung aspek teknis seperti jaringan dan sinkronisasi. Karena kemampuannya dalam

mengatur komunikasi antar proses secara efisien, MPI menjadi standar yang paling luas digunakan dalam komputasi paralel. Dalam konteks ini, MPI berperan penting dalam mengoordinasikan kerja antar proses (atau prosesor) agar setiap bagian tugas dapat diselesaikan secara terintegrasi dan tepat waktu

2.3 Gambaran Dataset

Dataset ini berisi informasi berbagai destinasi wisata dengan sejumlah atribut penting yang digunakan untuk analisis dan pengambilan keputusan. Dalam konteks SAW, sebagian besar atribut memiliki prinsip semakin besar semakin baik (**benefit**).

a. Location

Nama atau identitas destinasi wisata. Nilai ini biasanya berupa *string* (meskipun pada dataset ini bersifat sintetis atau acak).

Relevansi: digunakan sebagai penanda alternatif dalam proses pengambilan keputusan. Tidak berpengaruh langsung terhadap perhitungan, tetapi penting untuk interpretasi hasil ranking.

b. Visitors

Jumlah pengunjung yang datang ke destinasi tersebut dalam periode tertentu.

Relevansi: semakin banyak pengunjung menunjukkan popularitas dan daya tarik destinasi tersebut. Nilai yang lebih tinggi umumnya lebih diinginkan (**benefit**).

c. Rating

Nilai penilaian atau rating destinasi (biasanya dalam skala tertentu, misalnya 1–5).

Relevansi: mencerminkan tingkat kepuasan pengunjung. Semakin tinggi rating, semakin baik kualitas destinasi tersebut (**benefit**).

d. Revenue

Pendapatan yang dihasilkan dari destinasi wisata tersebut.

Relevansi: menunjukkan tingkat keberhasilan ekonomi dari destinasi. Nilai yang lebih tinggi dapat mengindikasikan fasilitas yang baik dan daya tarik tinggi (**benefit**).

e. Accommodation_Availability

Ketersediaan fasilitas akomodasi di sekitar destinasi (misalnya hotel, penginapan). Biasanya dalam bentuk jumlah atau indeks.

Relevansi: semakin tinggi ketersediaan akomodasi, semakin mudah bagi wisatawan untuk menginap, sehingga meningkatkan kenyamanan (**benefit**).

f. Distance

Jarak destinasi wisata dari titik keberangkatan pengguna atau pusat kota (misalnya dalam kilometer).

Relevansi: semakin dekat jarak destinasi, semakin kecil biaya dan waktu perjalanan yang dibutuhkan. Oleh karena itu, nilai jarak yang lebih kecil lebih diinginkan (*cost*).

g. Ticket_Price

Harga tiket masuk destinasi wisata.

Relevansi: harga tiket memengaruhi keterjangkauan destinasi bagi wisatawan. Semakin murah harga tiket, semakin baik dari sisi biaya (*cost*).

2.4 Pentingnya Paralelisasi dalam Data Besar

Pemrosesan paralel menjadi krusial ketika menangani perhitungan menggunakan metode SAW pada dataset berukuran besar, karena kompleksitas perhitungannya meningkat seiring bertambahnya jumlah data. Dalam metode SAW, setiap alternatif harus dihitung nilai preferensinya melalui proses normalisasi dan pembobotan, sehingga semakin banyak data, semakin besar pula beban komputasi yang harus ditangani. Dataset yang besar, seperti data apartemen dalam jumlah jutaan, akan memperlambat proses karena seluruh alternatif perlu diproses satu per satu.

Untuk mengatasi hal tersebut, pendekatan paralel seperti penggunaan MPI dapat diterapkan dengan cara membagi data ke beberapa prosesor agar perhitungan nilai preferensi dilakukan secara bersamaan. Dengan pembagian beban kerja ini, setiap prosesor menangani sebagian data, sehingga waktu komputasi dapat dipercepat secara signifikan. Hasil dari berbagai studi menunjukkan bahwa peningkatan jumlah prosesor berbanding lurus dengan percepatan waktu eksekusi, karena tugas-tugas besar dipecah menjadi bagian yang lebih kecil dan diproses secara paralel. Oleh karena itu, dalam konteks data berskala besar, paralelisasi tidak hanya meningkatkan efisiensi waktu, tetapi juga mendukung skalabilitas sistem, sehingga metode SAW tetap optimal untuk digunakan pada aplikasi berbasis *big data*.

BAB III

METODE

3.1 Cara Kerja Algoritma *Simple Additive Weighting* (SAW) Berbasis Scoring Model

Pada studi kasus rekomendasi destinasi wisata, setiap entitas data direpresentasikan sebagai alternatif yang memiliki sejumlah atribut atau kriteria penilaian. Kriteria-kriteria tersebut digunakan untuk mengevaluasi kualitas suatu destinasi wisata berdasarkan preferensi pengguna. Dalam penelitian ini, atribut yang digunakan meliputi location, visitor, rating, revenue, accommodation_availability, distance, ticket_price.

Setiap kriteria memiliki karakteristik yang berbeda, yaitu terdapat kriteria bertipe benefit (semakin besar nilainya semakin baik) dan kriteria bertipe cost (semakin kecil nilainya semakin baik). Sebagai contoh, rating dan fasilitas termasuk kriteria benefit, sedangkan harga tiket dan jarak termasuk kriteria cost.

Algoritma Simple Additive Weighting (SAW) digunakan untuk menentukan destinasi wisata terbaik dengan cara menghitung nilai preferensi dari setiap alternatif berdasarkan bobot kriteria yang telah ditentukan. Metode ini termasuk dalam kategori scoring model, di mana setiap alternatif akan diberikan skor akhir berdasarkan hasil penjumlahan terbobot dari seluruh kriteria.

Proses dalam metode SAW terdiri dari beberapa tahapan utama, yaitu pembentukan matriks keputusan, normalisasi nilai, perhitungan skor preferensi, hingga proses perankingan. Berikut adalah langkah-langkah rinci algoritma SAW berbasis scoring model dalam kasus pemilihan destinasi wisata:

1. Penentuan Alternatif dan Kriteria

Langkah awal dalam metode SAW adalah menentukan daftar alternatif destinasi wisata yang akan dievaluasi. Setiap destinasi wisata direpresentasikan sebagai satu baris data dalam sistem, yang berisi nilai dari masing-masing kriteria yang telah ditentukan. Alternatif ini dapat berupa berbagai pilihan tempat wisata, seperti pantai, gunung, taman rekreasi, maupun objek wisata lainnya.

Selanjutnya, ditentukan kriteria yang digunakan dalam proses penilaian. Dalam penelitian ini digunakan enam kriteria, yaitu location, visitor, rating, revenue, accommodation_availability, distance, dan ticket_price. Setiap kriteria memiliki peran penting dalam mencerminkan kualitas dan daya tarik suatu destinasi wisata, seperti tingkat popularitas, kemudahan akses, serta biaya yang harus dikeluarkan oleh pengunjung.

2. Penentuan Bobot Kriteria

Setiap kriteria diberikan bobot sesuai dengan tingkat kepentingannya terhadap keputusan akhir. Nilai bobot berada pada rentang 0 hingga 1, dengan ketentuan bahwa total seluruh bobot harus sama dengan 1. Proses pembobotan ini sangat penting karena akan mempengaruhi hasil akhir perhitungan preferensi.

Bobot dapat ditentukan berdasarkan preferensi pengguna, hasil survei, atau kebijakan tertentu dari sistem. Kriteria yang dianggap lebih penting akan diberikan bobot yang lebih besar, sehingga memiliki pengaruh yang lebih dominan dalam proses pengambilan keputusan.

3. Pembentukan Matriks Keputusan

Matriks keputusan dibentuk berdasarkan data alternatif dan kriteria yang telah ditentukan. Setiap baris pada matriks merepresentasikan satu destinasi wisata, sedangkan setiap kolom merepresentasikan satu kriteria.

Nilai pada setiap elemen matriks menunjukkan performa suatu destinasi wisata terhadap kriteria tertentu. Data ini dapat diperoleh dari hasil observasi, data statistik, atau sumber informasi lainnya. Matriks keputusan ini menjadi dasar utama dalam proses perhitungan metode SAW.

4. Normalisasi Matriks Keputusan

Karena setiap kriteria memiliki satuan dan skala yang berbeda, maka diperlukan proses normalisasi untuk menyamakan skala nilai agar dapat dibandingkan secara adil. Tanpa normalisasi, kriteria dengan nilai besar (misalnya revenue) dapat mendominasi perhitungan.

Proses normalisasi dilakukan sebagai berikut:

- Untuk kriteria benefit, nilai dibagi dengan nilai maksimum pada kolom tersebut
- Untuk kriteria cost, nilai minimum dibagi dengan nilai tersebut

Hasil dari proses ini adalah matriks normalisasi dengan nilai dalam rentang 0 hingga 1, sehingga seluruh kriteria berada pada skala yang sebanding.

5. Perhitungan Nilai Preferensi (Scoring Model)

Setelah proses normalisasi selesai, langkah berikutnya adalah menghitung nilai preferensi untuk setiap alternatif wisata. Nilai preferensi merupakan skor akhir yang menunjukkan tingkat kelayakan suatu destinasi.

$$V_i = \sum_{j=1}^7 w_j \cdot r_{ij}$$

Keterangan:

- V_i = nilai preferensi alternatif ke- i
- w_j = bobot kriteria ke- j
- r_{ij} = nilai normalisasi

Nilai ini diperoleh dari hasil penjumlahan terbobot seluruh kriteria, sehingga mencerminkan evaluasi menyeluruh terhadap setiap alternatif wisata.

6. Proses Perangkingan

Nilai preferensi yang telah diperoleh kemudian diurutkan dari nilai terbesar ke nilai terkecil. Proses ini bertujuan untuk menentukan urutan prioritas dari setiap destinasi wisata.

Alternatif dengan nilai preferensi tertinggi dianggap sebagai pilihan terbaik karena memiliki kombinasi nilai kriteria yang paling optimal sesuai dengan bobot yang telah ditentukan. Hasil perangkingan ini memudahkan pengguna dalam mengambil keputusan.

7. Analisis Hasil

Pada tahap ini dilakukan analisis terhadap hasil perangkingan untuk memahami pengaruh masing-masing kriteria terhadap keputusan akhir. Analisis ini penting untuk memastikan bahwa hasil yang diperoleh sudah sesuai dengan tujuan sistem.

Destinasi wisata dengan nilai preferensi tinggi umumnya memiliki karakteristik seperti:

- rating yang tinggi
- jumlah pengunjung (*visitor*) yang besar
- lokasi yang strategis
- ketersediaan akomodasi yang memadai
- serta nilai *distance* dan *ticket_price* yang relatif rendah

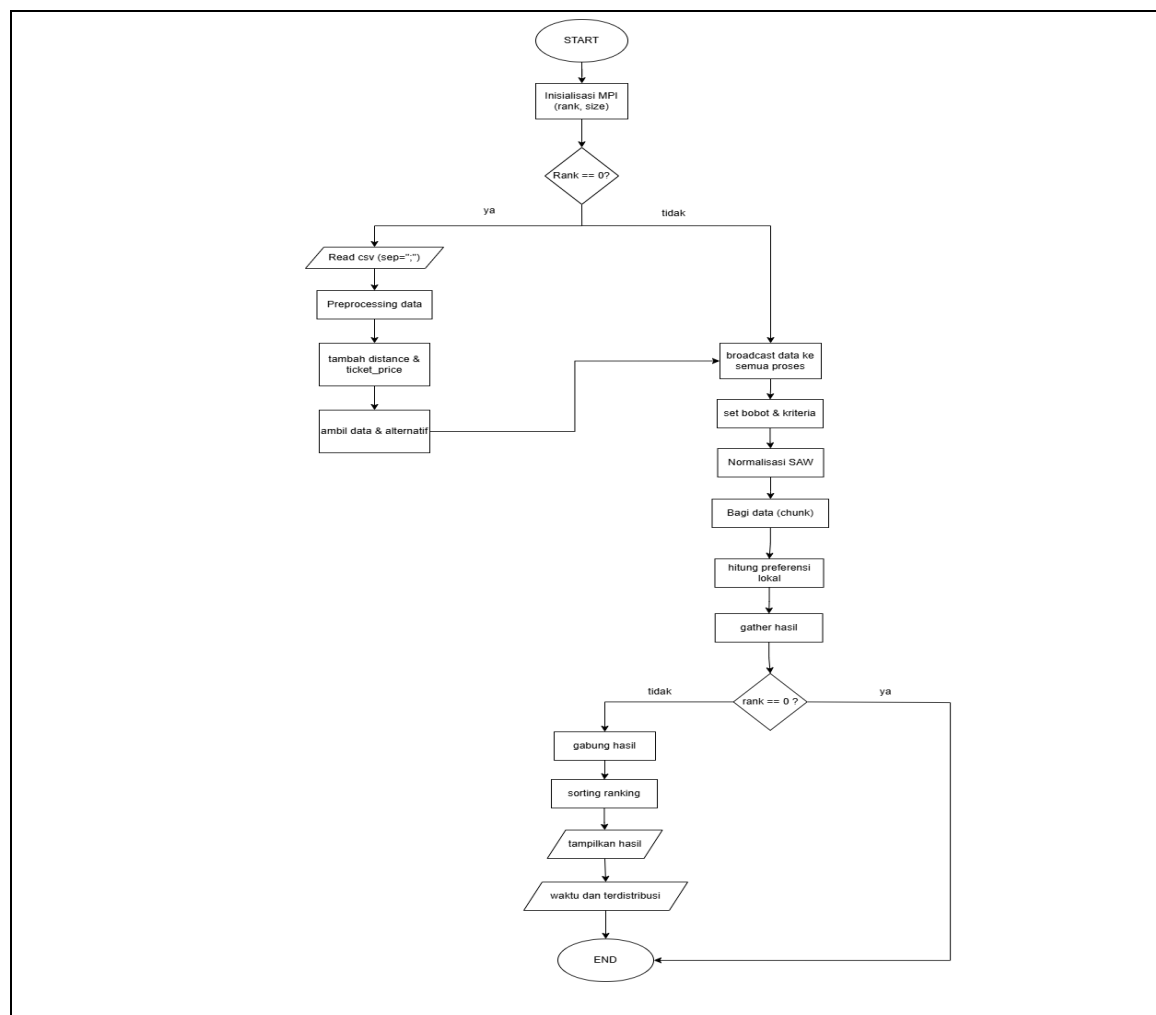
Selain itu, analisis juga dapat digunakan untuk mengevaluasi apakah bobot yang diberikan sudah mencerminkan prioritas yang diinginkan.

8. Output Sistem

Output akhir dari metode SAW adalah daftar destinasi wisata yang telah diurutkan berdasarkan nilai preferensi tertinggi. Sistem dapat menampilkan satu atau beberapa alternatif terbaik sebagai rekomendasi kepada pengguna. Hasil ini diharapkan dapat membantu pengguna dalam memilih destinasi wisata secara lebih objektif, cepat, dan sesuai dengan preferensi yang diinginkan.

Langkah-langkah di atas menjelaskan tentang cara kerja matematis dengan deskripsi algoritma *Simple Additive Weighting* (SAW) paralel berbasis scoring model menggunakan MPL. Beban komputasi tersebar merata, dan penggunaan scoring model serta operasi MPI membantu mempercepat proses serta memastikan hasil akhir yang akurat.

Berikut gambaran dalam bentuk diagram alir untuk tahap tahap dalam metode *Simple Additive Weighting* dapat dilihat pada gambar 1.



Flowchart tersebut menggambarkan alur kerja sistem rekomendasi menggunakan metode *Simple Additive Weighting* (SAW) yang dijalankan secara paralel dengan Message Passing Interface (MPI). Proses dimulai dengan inisialisasi MPI untuk mengetahui rank dan jumlah proses yang digunakan. Selanjutnya sistem memeriksa apakah proses memiliki rank 0 sebagai *master process*. Jika rank bernilai 0, sistem membaca data dari file CSV, melakukan *preprocessing* data, menambahkan atribut seperti *distance* dan “*ticket_price*”, lalu mengambil data alternatif yang akan diproses.

Data tersebut kemudian dibroadcast ke seluruh proses agar setiap proses memiliki data yang sama. Setelah itu dilakukan pengaturan bobot dan jenis kriteria, kemudian data

dinormalisasi menggunakan metode SAW. Data yang telah dinormalisasi dibagi menjadi beberapa bagian (chunk) sesuai jumlah proses MPI sehingga setiap proses dapat menghitung nilai preferensi lokal secara paralel.

Hasil perhitungan dari setiap proses kemudian dikumpulkan kembali menggunakan gather. Selanjutnya master process menggabungkan seluruh hasil, melakukan pengurutan ranking berdasarkan nilai preferensi tertinggi, dan menampilkan hasil akhir. Selain itu, sistem juga menampilkan waktu eksekusi untuk melihat performa komputasi terdistribusi sebelum program selesai dijalankan.

BAB IV

PENGUJIAN DAN PEMBAHASAN

4.1 Eksperimen pada dataset

1. *Source code*

```
from mpi4py import MPI
import pandas as pd
import random

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

start_time = MPI.Wtime()

if rank == 0:
    df = pd.read_csv("tourism_dataset_1juta.csv", sep=";")

    df.columns = df.columns.str.strip().str.lower()

    df = df.rename(columns={
        "accommodation_available": "accommodation_availability"
    })

    df["accommodation_availability"] =
df["accommodation_availability"].map({
    "Yes": 1,
    "No": 0
})

    df["distance"] = [random.randint(1, 50) for _ in range(len(df))]
    df["ticket_price"] = [random.randint(10000, 100000) for _ in
range(len(df))]

    data = df[[
        "visitors",
        "rating",
        "revenue",
        "accommodation_availability",
        "distance",
        "ticket_price"
    ]].values.tolist()
    for j in range(len(data[0])):
        kolom = [baris[j] for baris in data]
        nilai_min = min(kolom)
        if nilai_min <= 0:
            shift = abs(nilai_min) + 1
            for i in range(len(data)):
                data[i][j] = data[i][j] + shift

    alternatif = (
        df["location"] +
        " (" + df["country"] + ") " +
        " - " + df["category"]
    ).tolist()

else:
    data = None
```

```

alternatif = None

data = comm.bcast(data, root=0)
alternatif = comm.bcast(alternatif, root=0)

w = [0.25, 0.2, 0.2, 0.15, 0.1, 0.1]
kriteria = ["benefit", "benefit", "benefit", "benefit", "cost",
"cost"]
kolom_data = list(zip(*data))
normalisasi = []

for j in range(len(kolom_data)):
    kolom = kolom_data[j]

    if kriteria[j] == "benefit":
        max_val = max(kolom)
        normalisasi.append([data[i][j] / max_val for i in
range(len(data))])
    else:
        min_val = min(kolom)
        normalisasi.append([min_val / data[i][j] for i in
range(len(data))])

normalisasi = list(zip(*normalisasi))

rows = len(normalisasi)
chunk = rows // size

start = rank * chunk
end = start + chunk if rank != size - 1 else rows

local_data = normalisasi[start:end]

info_proses = comm.gather((rank, len(local_data)), root=0)

local_pref = []

for row in local_data:
    nilai = 0
    for j in range(len(w)):
        nilai += w[j] * row[j]
    local_pref.append(nilai)

all_pref = comm.gather(local_pref, root=0)

if rank == 0:
    preferensi = []
    for part in all_pref:
        preferensi.extend(part)

    hasil = list(zip(alternatif, preferensi))
    hasil.sort(key=lambda x: x[1], reverse=True)

    print("\n=== RANKING SEMUA DATA ===\n")
    for i, (nama, nilai) in enumerate(hasil[:10], start=1): # tampil
10 teratas
        print(f"{i}. {nama} -> {nilai:.4f}")

    end_time = MPI.Wtime()

    print("\nWaktu eksekusi:", end_time - start time, "detik")

```

```
print("Total data:", len(data))

print("\nDistribusi proses:")
for r, jumlah in sorted(info_proses):
    print(f"Proses {r} mengerjakan {jumlah} data")
```

2. Analisis Data

```
from mpi4py import MPI
import pandas as pd
import random
```

Script di atas digunakan untuk mengimpor beberapa *library* penting yang diperlukan dalam program. *Library* *mpi4py* digunakan untuk menerapkan konsep *parallel processing* dengan metode MPI (*Message Passing Interface*). *Library* *pandas* digunakan untuk membaca dan mengolah data berbentuk tabel atau dataset. Sedangkan *library* *random* digunakan untuk menghasilkan nilai acak yang nantinya dipakai pada atribut jarak “distance” dan harga tiket “ticket_price”.

```
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
```

Script di atas berfungsi untuk menginisialisasi komunikasi MPI. Variabel “comm” digunakan sebagai objek komunikasi global antar proses. Variabel *rank* digunakan untuk mengetahui identitas atau nomor proses yang sedang berjalan. Sedangkan variabel *size* digunakan untuk mengetahui jumlah total proses yang aktif saat program dijalankan.

```
start_time = MPI.Wtime()
```

Script di atas digunakan untuk mencatat waktu awal eksekusi program menggunakan fungsi “MPI.Wtime()”. Waktu ini nantinya dipakai untuk menghitung total lama proses komputasi paralel.

```
if rank == 0:
    df = pd.read_csv("tourism_dataset_1juta.csv", sep=";")
```

Script di atas digunakan pada proses dengan “rank = 0” yang bertugas membaca *dataset* dari file CSV. Dataset dibaca menggunakan fungsi “read_csv()” dari *library* *pandas* dengan pemisah data berupa tanda titik koma (;).

```
df.columns = df.columns.str.strip().str.lower()
```

Script di atas digunakan untuk membersihkan nama kolom pada dataset. Fungsi “strip()” digunakan untuk menghapus spasi berlebih, sedangkan “lower()” digunakan untuk mengubah seluruh nama kolom menjadi huruf kecil agar lebih konsisten dan mudah diproses.

```
df = df.rename(columns={
    "accommodation_available": "accommodation_availability"
})
```

Script di atas digunakan untuk mengganti nama kolom dari “accommodation_available” menjadi “accommodation_availability” agar penamaan atribut lebih sesuai dan konsisten pada proses pengolahan data berikutnya

```
df["accommodation_availability"] =
df["accommodation_availability"].map({
    "Yes": 1,
    "No": 0
})
```

Script di atas digunakan untuk mengubah data kategorikal menjadi data numerik. Nilai “Yes” diubah menjadi 1, sedangkan nilai “No” diubah menjadi 0. Proses ini dilakukan agar data dapat digunakan dalam proses perhitungan metode pengambilan keputusan.

```
df["distance"] = [random.randint(1, 50) for _ in range(len(df))]
df["ticket_price"] = [random.randint(10000, 100000) for _ in
range(len(df))]
```

Script di atas digunakan untuk menambahkan atribut baru berupa jarak wisata “distance” dan harga tiket “ticket_price”. Nilai kedua atribut tersebut dihasilkan secara acak menggunakan fungsi “random.randint()”. Nilai jarak dibuat antara 1 sampai 50 kilometer. Nilai harga tiket dibuat antara Rp10.000 sampai Rp100.000.

```
data = df[[
    "visitors",
    "rating",
    "revenue",
    "accommodation_availability",
    "distance",
    "ticket_price"
]].values.tolist()
```

Script di atas digunakan untuk mengambil atribut yang akan diproses dalam metode pengambilan keputusan, yaitu jumlah pengunjung (*visitors*), rating wisata (*rating*), pendapatan (*revenue*), ketersediaan penginapan (*accommodation availability*), jarak (*distance*), dan harga tiket (*ticket price*). Data dari atribut tersebut kemudian diubah ke dalam bentuk *list* menggunakan fungsi `values.tolist()` agar lebih mudah diproses pada sistem komputasi paralel MPI, terutama saat proses distribusi dan pengolahan data dilakukan oleh beberapa proses secara bersamaan.

```
for j in range(len(data[0])):

    kolom = [baris[j] for baris in data]
    nilai_min = min(kolom)

    if nilai_min <= 0:
        shift = abs(nilai_min) + 1
```

```
for i in range(len(data)):

    data[i][j] = data[i][j] + shift
```

Script di atas digunakan untuk memastikan seluruh nilai pada setiap kolom data bernilai positif. Program memeriksa setiap kolom, kemudian mencari nilai terkecil pada kolom tersebut. Jika ditemukan nilai yang kurang dari atau sama dengan nol, maka dilakukan penambahan (*shift*) pada seluruh nilai di kolom tersebut sebesar $|\text{nilai minimum}| + 1$. Dengan cara ini, nilai terkecil akan berubah menjadi 1 dan semua nilai lainnya ikut bergeser menjadi positif. Proses ini sering dilakukan sebagai tahap praproses data sebelum menggunakan metode yang mensyaratkan seluruh data bernilai positif.

```
alternatif = (
    df["location"] +
    " (" + df["country"] + ")" +
    " - " + df["category"]
).tolist()
```

Script di atas digunakan untuk membuat nama alternatif wisata berdasarkan gabungan lokasi, negara, dan kategori wisata. Data kemudian diubah menjadi list agar mudah dipasangkan dengan nilai preferensi akhir.

```
data = comm.bcast(data, root=0)
alternatif = comm.bcast(alternatif, root=0)
```

Script di atas digunakan untuk mengirimkan data dari proses utama ($\text{root} = 0$) ke seluruh proses lainnya menggunakan metode broadcast. Dengan demikian, setiap proses memiliki salinan data yang sama untuk diproses secara paralel.

```
w = [0.25, 0.2, 0.2, 0.15, 0.1, 0.1]
kriteria = ["benefit", "benefit", "benefit", "benefit", "cost",
"cost"]
```

Script di atas digunakan untuk menentukan bobot setiap kriteria serta jenis kriterianya dalam proses perhitungan metode SAW. Setiap kriteria memiliki bobot yang menunjukkan tingkat kepentingannya, yaitu Visitors sebesar 0.25, Rating sebesar 0.20, Revenue sebesar 0.20, Accommodation Availability sebesar 0.15, Distance sebesar 0.10, dan Ticket Price sebesar 0.10. Selain itu, setiap kriteria juga diklasifikasikan menjadi dua jenis, yaitu benefit dan cost. Kriteria benefit berarti semakin besar nilainya semakin baik, sedangkan cost berarti semakin kecil nilainya semakin baik. Dalam kasus ini, kriteria distance dan ticket_price termasuk jenis cost, sementara kriteria lainnya termasuk jenis benefit.

```
kolom_data = list(zip(*data))
normalisasi = []
```

Script di atas digunakan untuk memisahkan data berdasarkan kolom agar proses normalisasi lebih mudah dilakukan. Dengan menggunakan `zip(*data)`, data yang awalnya

berbentuk baris akan diputar menjadi kolom sehingga setiap kriteria dapat diproses secara terpisah.

```
if kriteria[j] == "benefit":
    max_val = max(kolom)
    normalisasi.append([data[i][j] / max_val for i in range(len(data))])
```

Script di atas digunakan untuk menjelaskan untuk kriteria benefit, proses normalisasi dilakukan dengan membagi setiap nilai dengan nilai maksimum pada kolom tersebut. Hal ini bertujuan agar nilai terbesar menjadi 1, sedangkan nilai lainnya berada di bawah 1 sesuai proporsinya.

```
else:
    min_val = min(kolom)
    normalisasi.append([min_val/ data[i][j] for i in range(len(data))])
```

Script di atas digunakan untuk menjelaskan kriteria cost, proses normalisasi dilakukan dengan membagi nilai minimum dengan setiap nilai pada kolom tersebut. Tujuannya adalah agar nilai yang lebih kecil menjadi lebih baik dan memiliki nilai normalisasi yang lebih besar.

```
rows = len(normalisasi)
chunk = rows // size
start = rank * chunk
end = start + chunk if rank != size - 1 else rows
local_data = normalisasi[start:end]
```

Script di atas digunakan untuk membagi data normalisasi ke setiap proses MPI. Setiap proses akan mengerjakan sebagian data sesuai nomor proses (rank) masing-masing sehingga proses komputasi menjadi lebih cepat.

```
info_proses = comm.gather((rank, len(local_data)), root=0)
```

Script di atas digunakan untuk mengumpulkan informasi jumlah data yang dikerjakan oleh setiap proses ke proses utama (root).

```
local_pref = []
for row in local_data:
    nilai = 0
    for j in range(len(w)):
        nilai += w[j] * row[j]
    local_pref.append(nilai)
```

Script di atas digunakan untuk menghitung nilai preferensi menggunakan metode Simple Additive Weighting (SAW). Nilai akhir diperoleh dari penjumlahan hasil perkalian antara bobot kriteria dengan nilai normalisasi.

```
all_pref = comm.gather(local_pref, root=0)
```

Script di atas digunakan untuk mengumpulkan seluruh hasil perhitungan preferensi dari setiap proses menuju proses utama (root).

```
hasil = list(zip(alternatif, preferensi))
hasil.sort(key=lambda x: x[1], reverse=True)
```

Script di atas digunakan untuk menggabungkan nama alternatif wisata dengan nilai preferensi, kemudian mengurutkannya dari nilai terbesar ke terkecil sehingga diperoleh peringkat wisata terbaik.

```
for i, (nama, nilai) in enumerate(hasil[:10], start=1):  
    print(f"{i}. {nama} -> {nilai:.4f}")
```

Script di atas digunakan untuk menampilkan 10 data wisata terbaik berdasarkan nilai preferensi tertinggi hasil perhitungan metode SAW.

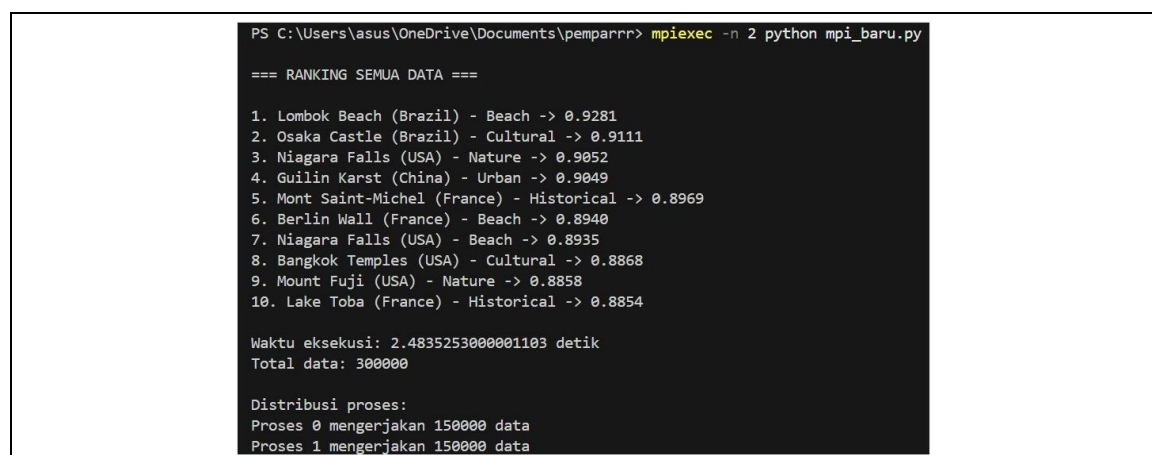
```
end_time = MPI.Wtime()  
print("\nWaktu eksekusi:", end_time - start_time, "detik")
```

Script di atas digunakan untuk mencatat waktu akhir eksekusi program sekaligus langsung menampilkan hasil perhitungan waktu eksekusi. Nilai `end_time` diambil dari `MPI.Wtime()` sebagai waktu selesai program berjalan, kemudian dikurangi dengan `start_time` untuk mendapatkan total durasi eksekusi program. Hasil pengurangan tersebut langsung dicetak menggunakan `print`, sehingga program sekaligus menghitung dan menampilkan waktu eksekusi dalam satu bagian.

```
for r, jumlah in sorted(info_proses):  
    print(f"Proses {r} mengerjakan {jumlah} data")
```

Script di atas digunakan untuk menampilkan informasi pembagian data pada setiap proses MPI, sehingga dapat diketahui distribusi pekerjaan antar proses dalam komputasi paralel.

3. Output



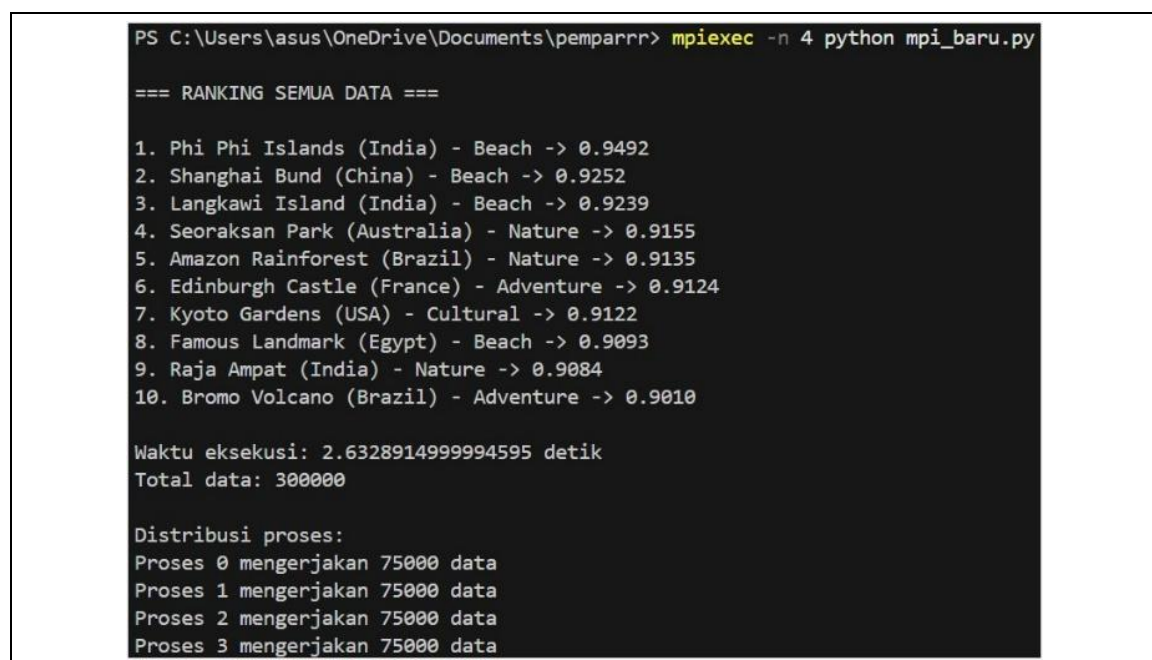
```
PS C:\Users\asus\OneDrive\Documents\pemparr> mpiexec -n 2 python mpi_baru.py  
  
=== RANKING SEMUA DATA ===  
  
1. Lombok Beach (Brazil) - Beach -> 0.9281  
2. Osaka Castle (Brazil) - Cultural -> 0.9111  
3. Niagara Falls (USA) - Nature -> 0.9052  
4. Guilin Karst (China) - Urban -> 0.9049  
5. Mont Saint-Michel (France) - Historical -> 0.8969  
6. Berlin Wall (France) - Beach -> 0.8940  
7. Niagara Falls (USA) - Beach -> 0.8935  
8. Bangkok Temples (USA) - Cultural -> 0.8868  
9. Mount Fuji (USA) - Nature -> 0.8858  
10. Lake Toba (France) - Historical -> 0.8854  
  
Waktu eksekusi: 2.4835253000001103 detik  
Total data: 300000  
  
Distribusi proses:  
Proses 0 mengerjakan 150000 data  
Proses 1 mengerjakan 150000 data
```

Gambar tersebut menampilkan hasil eksekusi program paralel menggunakan MPI (*Message Passing Interface*) untuk melakukan proses pemeringkatan data destinasi wisata. *Output* program memperlihatkan 10 data dengan nilai skor tertinggi hasil perhitungan sistem, di mana setiap data terdiri dari nama destinasi, kategori wisata, dan nilai skor akhir.

Semakin tinggi nilai skor yang diperoleh, maka semakin tinggi pula posisi *ranking* destinasi tersebut. Pada hasil tersebut, “Lombok Beach” memperoleh nilai tertinggi yaitu 0,9281 sehingga menempati peringkat pertama.

Selain menampilkan hasil *ranking*, program juga menunjukkan performa pemrosesan paralel yang dilakukan. Total data yang diproses mencapai 300.000 data dengan waktu eksekusi sekitar 2,48 detik. Hal ini menunjukkan bahwa implementasi MPI mampu memproses data dalam jumlah besar dengan waktu yang relatif singkat. Distribusi proses juga terlihat pada bagian bawah *output*, di mana proses 0 dan proses 1 masing-masing mengerjakan 150.000 data. Pembagian beban kerja secara merata ini memungkinkan proses komputasi berjalan secara paralel sehingga mempercepat waktu pemrosesan dibandingkan metode *sequential*.

Dari hasil tersebut dapat disimpulkan bahwa penggunaan MPI efektif dalam meningkatkan efisiensi pemrosesan data skala besar. Dengan membagi data ke beberapa proses, beban komputasi menjadi lebih ringan dan waktu eksekusi dapat ditekan, sehingga sistem tetap optimal meskipun jumlah data yang diproses sangat besar.



```
PS C:\Users\asus\OneDrive\Documents\pemparr> mpiexec -n 4 python mpi_baru.py

=== RANKING SEMUA DATA ===

1. Phi Phi Islands (India) - Beach -> 0.9492
2. Shanghai Bund (China) - Beach -> 0.9252
3. Langkawi Island (India) - Beach -> 0.9239
4. Seoraksan Park (Australia) - Nature -> 0.9155
5. Amazon Rainforest (Brazil) - Nature -> 0.9135
6. Edinburgh Castle (France) - Adventure -> 0.9124
7. Kyoto Gardens (USA) - Cultural -> 0.9122
8. Famous Landmark (Egypt) - Beach -> 0.9093
9. Raja Ampat (India) - Nature -> 0.9084
10. Bromo Volcano (Brazil) - Adventure -> 0.9010

Waktu eksekusi: 2.6328914999994595 detik
Total data: 300000

Distribusi proses:
Proses 0 mengerjakan 75000 data
Proses 1 mengerjakan 75000 data
Proses 2 mengerjakan 75000 data
Proses 3 mengerjakan 75000 data
```

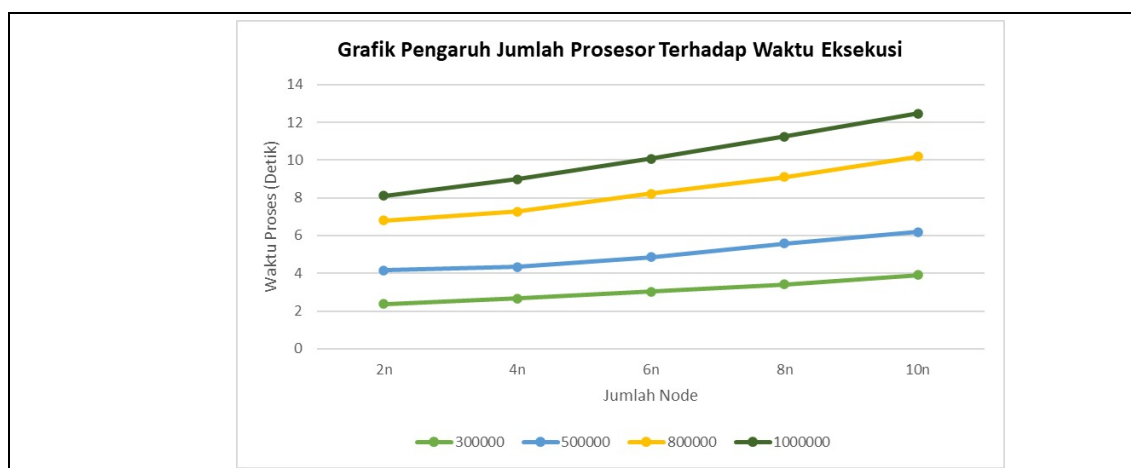
Gambar tersebut menampilkan hasil pengujian program paralel menggunakan MPI (*Message Passing Interface*) dengan 4 proses untuk melakukan pemeringkatan data destinasi wisata. Program dijalankan menggunakan perintah “`mpiexec -n 4`”, yang berarti beban komputasi dibagi ke dalam empat proses berbeda agar pemrosesan data dapat dilakukan secara paralel. Hasil *output* menampilkan 10 destinasi wisata dengan nilai skor

tertinggi berdasarkan hasil perhitungan sistem. Pada hasil tersebut, “Phi Phi Islands” memperoleh nilai tertinggi sebesar 0,9492 sehingga berada pada peringkat pertama.

Selain hasil *ranking*, *output* juga menunjukkan informasi performa sistem. Total data yang diproses mencapai 300.000 data dengan waktu eksekusi sekitar 2,63 detik. Meskipun jumlah data yang diproses sangat besar, waktu eksekusi tetap relatif cepat karena adanya mekanisme paralelisme menggunakan MPI. Pada bagian distribusi proses terlihat bahwa setiap proses memperoleh pembagian data yang merata, yaitu masing-masing proses mengerjakan 75.000 data. Pembagian beban kerja ini membuat proses komputasi dapat berjalan secara bersamaan sehingga meningkatkan efisiensi pemrosesan.

Dari hasil tersebut dapat disimpulkan bahwa implementasi MPI berhasil membantu mempercepat pengolahan data dalam jumlah besar melalui distribusi pekerjaan ke beberapa proses. Dengan penggunaan 4 proses, sistem mampu menjaga performa komputasi tetap optimal dan stabil meskipun kompleksitas pengolahan data meningkat.

4. Pengaruh Jumlah Prosesor Terhadap Waktu Eksekusi



Grafik menunjukkan hubungan antara jumlah prosesor (node) terhadap waktu eksekusi metode *Simple Additive Weighting* (SAW). Pengujian dilakukan menggunakan beberapa variasi jumlah data, yaitu 300.000, 500.000, 800.000, dan 1.000.000 data. Tujuan pengujian ini adalah untuk mengetahui bagaimana perubahan jumlah node memengaruhi performa proses perhitungan metode SAW.

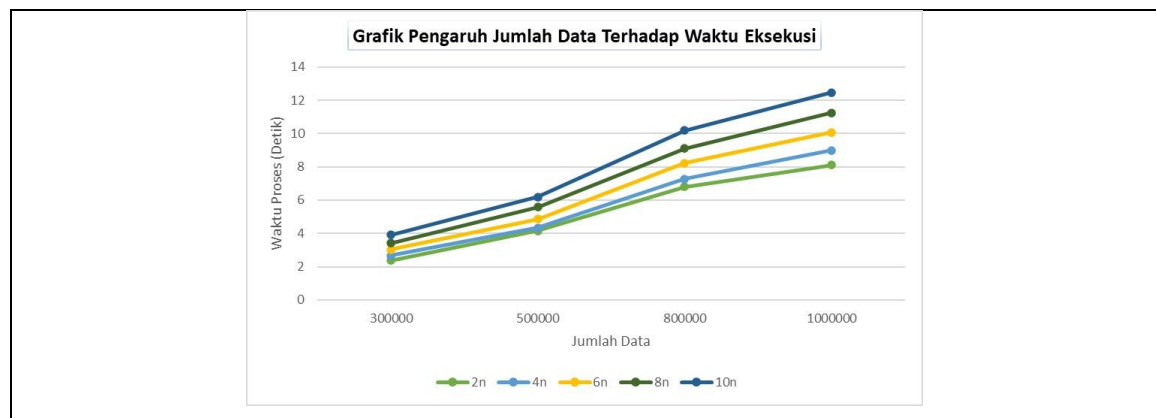
Berdasarkan grafik, terlihat bahwa setiap penambahan jumlah node dari 2n hingga 10n menyebabkan waktu proses eksekusi mengalami peningkatan pada seluruh jumlah data. Pada data sebanyak 300.000, waktu eksekusi meningkat secara bertahap dari sekitar 2 detik hingga mendekati 4 detik. Pada data 500.000, waktu proses berada pada kisaran 4–6 detik. Sementara itu, data 800.000 dan 1.000.000 menunjukkan waktu eksekusi yang

lebih tinggi dibandingkan jumlah data lainnya, dengan peningkatan paling besar terjadi pada data 1.000.000 yang mencapai lebih dari 12 detik pada penggunaan 10 node.

Selain itu, semakin besar jumlah data yang diproses, maka garis pada grafik juga semakin tinggi. Hal ini menunjukkan bahwa ukuran data sangat memengaruhi waktu komputasi metode SAW. Penambahan node memang membantu pembagian proses, namun komunikasi dan sinkronisasi antar node juga menambah beban proses sehingga waktu eksekusi tetap meningkat.

Berdasarkan hasil pengujian tersebut, dapat disimpulkan bahwa jumlah data memiliki pengaruh besar terhadap waktu eksekusi metode SAW. Semakin besar data yang diproses dan semakin banyak node yang digunakan, maka waktu proses cenderung meningkat. Oleh karena itu, diperlukan pengelolaan distribusi proses yang optimal agar penggunaan banyak node dapat memberikan performa yang lebih efisien.

5. Pengaruh Jumlah Data Terhadap Waktu Eksekusi



Grafik menunjukkan pengaruh jumlah data terhadap waktu eksekusi metode *Simple Additive Weighting* (SAW) dengan variasi jumlah node, yaitu 2n, 4n, 6n, 8n, dan 10n. Pengujian dilakukan untuk melihat seberapa besar peningkatan waktu proses ketika jumlah data yang diproses semakin banyak.

Berdasarkan grafik, terlihat bahwa seluruh garis mengalami kenaikan seiring bertambahnya jumlah data dari 300.000 hingga 1.000.000 data. Pada penggunaan 2 node, waktu proses meningkat dari sekitar 2 detik menjadi lebih dari 7 detik. Pada penggunaan 4 node dan 6 node, peningkatan waktu eksekusi juga terjadi secara bertahap. Sementara itu, penggunaan 8 node dan 10 node menghasilkan waktu proses paling tinggi dibandingkan node lainnya, terutama pada data berjumlah 1.000.000 yang mencapai lebih dari 10 detik.

Kenaikan waktu eksekusi ini terjadi karena semakin banyak data yang diproses, maka proses normalisasi, pembobotan, dan perangkingan dalam metode SAW membutuhkan komputasi yang lebih besar. Selain itu, penggunaan jumlah node yang lebih banyak menyebabkan adanya tambahan proses komunikasi antar node sehingga waktu eksekusi ikut bertambah.

Berdasarkan hasil grafik tersebut, dapat disimpulkan bahwa jumlah data berbanding lurus dengan waktu eksekusi metode SAW. Semakin besar jumlah data yang digunakan, maka semakin lama waktu proses yang dibutuhkan. Perbedaan jumlah node juga memengaruhi performa sistem, namun peningkatan jumlah node belum tentu menghasilkan waktu proses yang lebih cepat apabila overhead komunikasi antar node semakin besar.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penerapan metode Simple Additive Weighting (SAW) pada sistem pendukung keputusan, dapat disimpulkan bahwa metode SAW mampu digunakan secara efektif dalam proses pengambilan keputusan multikriteria. Metode ini bekerja dengan melakukan normalisasi nilai pada setiap kriteria, kemudian menghitung nilai preferensi berdasarkan bobot yang telah ditentukan sehingga menghasilkan proses perangkingan alternatif secara sistematis dan objektif.

Penerapan metode SAW mempermudah proses penentuan alternatif terbaik karena seluruh kriteria yang digunakan dapat dihitung dan dibandingkan secara terstruktur. Hasil perangkingan yang diperoleh menunjukkan bahwa alternatif dengan nilai preferensi tertinggi dapat dijadikan rekomendasi utama dalam pengambilan keputusan. Dengan demikian, metode SAW mampu membantu pengguna dalam memperoleh keputusan yang lebih cepat, tepat, dan akurat sesuai dengan kebutuhan yang telah ditentukan.

Selain itu, integrasi metode SAW dengan komputasi paralel memberikan peningkatan efisiensi pada proses pengolahan data, terutama ketika jumlah data alternatif dan kriteria semakin besar. Komputasi paralel memungkinkan proses perhitungan dilakukan secara bersamaan sehingga waktu pemrosesan menjadi lebih singkat dibandingkan perhitungan secara *sequential*. Hal ini menunjukkan bahwa penerapan komputasi paralel dapat meningkatkan performa sistem pendukung keputusan berbasis SAW, baik dari sisi kecepatan maupun efisiensi pemrosesan data.

Secara keseluruhan, metode SAW terbukti menjadi salah satu metode yang sederhana, mudah diterapkan, dan efektif dalam menyelesaikan permasalahan pengambilan keputusan multikriteria. Dengan adanya proses perangkingan yang jelas dan terukur, metode ini dapat digunakan sebagai pendukung dalam berbagai bidang yang membutuhkan pemilihan alternatif terbaik berdasarkan sejumlah kriteria tertentu.

5.1 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan penelitian selanjutnya. Metode *Simple Additive Weighting* (SAW) dapat dikombinasikan dengan metode pengambilan keputusan lainnya seperti TOPSIS, AHP, maupun *Weighted Product* agar dapat dilakukan perbandingan hasil dan performa metode secara lebih mendalam. Selain itu, penggunaan jumlah data alternatif

dan kriteria yang lebih besar juga disarankan agar pengujian sistem, khususnya pada penerapan komputasi paralel, dapat memberikan hasil yang lebih optimal dan realistis terhadap kondisi sebenarnya.

Penentuan bobot pada setiap kriteria sebaiknya dilakukan dengan pendekatan yang lebih objektif, misalnya melalui survei, observasi, atau metode pembobotan tertentu sehingga hasil perankingan yang diperoleh menjadi lebih akurat dan sesuai dengan kebutuhan pengguna. Pengembangan sistem dalam bentuk aplikasi berbasis web maupun mobile juga dapat dilakukan agar sistem pendukung keputusan lebih mudah diakses dan digunakan secara praktis. Selain itu, optimalisasi komputasi paralel perlu terus dikembangkan untuk meningkatkan efisiensi waktu pemrosesan data, terutama ketika sistem digunakan untuk menangani data dalam jumlah besar dan kompleksitas yang lebih tinggi.

DAFTAR PUSTAKA

- [1] R. T. Subagio, M. T. Abdullah, dan Jaenudin, "Penerapan Metode SAW (Simple Additive Weighting) dalam Sistem Pendukung Keputusan untuk Menentukan Penerima Beasiswa," *Jurnal Ilmiah Intech: Information Technology Journal of UMUS*, vol. 2, no. 1, hlm. 31–40, Mei 2020.
- [2] Z. Vikki, "Implementasi Algoritma Simple Additive Weighting (SAW) Untuk Decision Support System (DSS) Penilaian Kinerja Karyawan Di Universitas Islam Kebangsaan Indonesia," *JETI (Jurnal Elektronika dan Teknologi Informasi)*, vol. 3, no. 2, hlm. 7–12, Sep. 2022.
- [3] Refiza, "Penerapan Metode Simple Additive Weighting (SAW) Untuk Seleksi Tenaga Kerja," dalam *Prosiding SEMNASTEK UISU 2019*, Medan, 2019, hlm. 1–6.
- [4] M. S. R. Damanik, Suendri, dan A. Ikhwan, "Implementasi Metode SAW dan TOPSIS Dalam Sistem Pendukung Keputusan Untuk Seleksi Calon Penegak Saka Bhayangkara," *RESOLUSI: Rekayasa Teknik Informatika dan Informasi*, vol. 5, no. 1, hlm. 21–29, Sep. 2024.
- [5] M. Hatta, "Implementasi Metode Simple Additive Weighting Pada Sistem DSS Seleksi Penerimaan Beasiswa Perguruan Tinggi," *Jurnal Ilmiah Intech: Information Technology Journal of UMUS*, vol. 2, no. 1, hlm. 31–40, Mei 2020.
- [6] Desyanti, Mesran, dan E. W. Ambarsari, "Implementasi Metode Simple Additive Weighting (SAW) untuk Sistem Pendukung Keputusan Pemilihan Dosen Favorit Mahasiswa," *Journal of Informatics Management and Information Technology (JIMAT)*, vol. 4, no. 3, hlm. 92–97, Jul. 2024.
- [7] Supiandi, I. T. Kusnadi, dan W. Kusnadi, "Penerapan Simple Additive Weighting (SAW) Untuk Sistem Penunjang Keputusan Kenaikan Jabatan Karyawan," *Jurnal Swabumi*, vol. 10, no. 2, hlm. 107–114, Sep. 2022.
- [8] K. M. Sukiakhy, C. V. R. Jummi, dan A. R. Utami, "Implementasi Metode SAW Dalam Sistem Pendukung Keputusan Pemilihan Karyawan Terbaik Pada PT. Cindyani Tiwi Lestari," *Jurnal Sistem Informasi dan Sistem Komputer (SIMKOM)*, vol. 7, no. 1, hlm. 1–9, Jan. 2022.
- [9] M. N. Gayo dan A. J. P. Sibarani, "Sistem Rekomendasi Penerima Zakat Untuk Mustahiq Dengan Metode Simple Additive Weighting (SAW)," *Jurnal Komputer Terapan*, vol. 7, no. 1, hlm. 84–99, Mei 2021.
- [10] P. Zandi, M. Ajalli, dan N. S. Ekhtiyati, "An extended simple additive weighting decision support system with application in the food industry," *Decision Analytics Journal*, vol. 14, hlm. 1–16, 2025, Art. no. 100553.